ELO320 - Estructuras de Datos y Algoritmos
Tipos de Datos Abstracts
Pilas, Listas Circulares y Colas

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

2do Semestre 2022

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

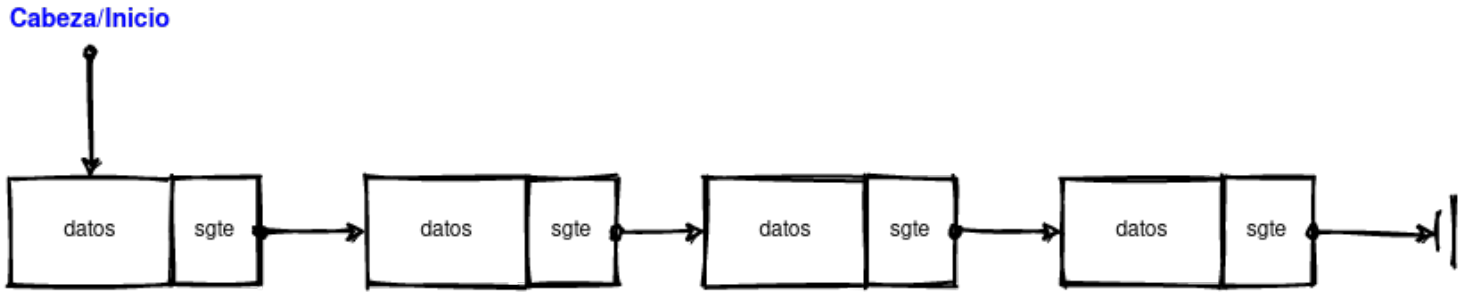
Lista Simple Enlazada: Inicializar

Las listas, en su versión más básica, dependen de un puntero al inicio de la lista. Éste suele llamarse *header*, *cabecera* o *inicio*.

C

```c
#include<stdio.h>

typedef struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo enalce.
} nodo_t;

int main(int argc, char **argv){

        nodo_t * inicio = NULL;
        // Inicializamos la lista, con su puntero de inicio apuntando a NULL
        // Equivalente a lista vacía []

        return 0;
}
```

Lista Simple Enlazada: Inicializar



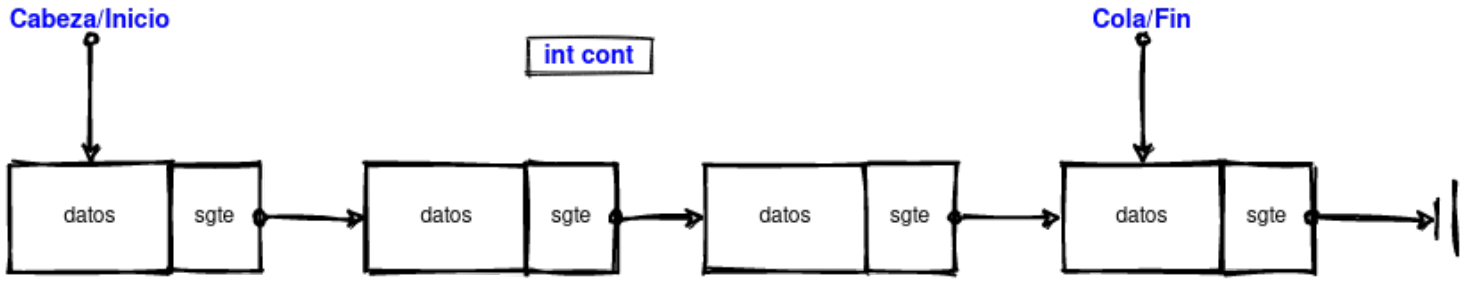
Lista Enlazada PRO

Existen otras implementaciones de listas enlazadas simples, donde se incluyen más punteros y variables de control.

C

```c
#include<stdio.h>

typedef struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo enalce.
} nodo_t;

int main(int argc, char **argv){

        nodo_t * inicio = NULL; //pueden usar un struct para estos datos
        nodo_t * final = NULL;
        int largo = 0;
        // Equivalente a lista vacía []
        // ¿Para que sirve esta nueva forma?
        // Implemente las funciones de ésta clase considerando esta forma.

        return 0;
}
```

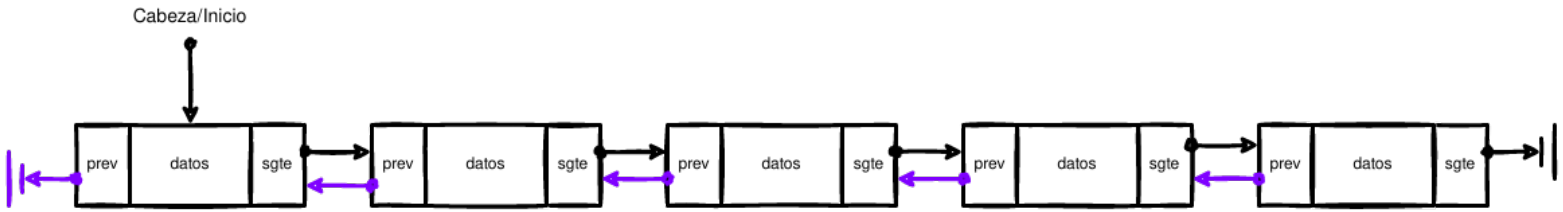
Lista Enlazada PRO



Lista Doblemente Enlazada

C

```c
#include<stdio.h>

typedef struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo siguiente, enalce.
        struct nodo *prev; // puntero a un nodo previo, enalce.
} nodo_t;

int main(int argc, char **argv){

        nodo_t * inicio = NULL;
        // Inicializamos la lista, con su puntero de inicio apuntando a NULL
        // Equivalente a lista vacía []

        return 0;
}
```



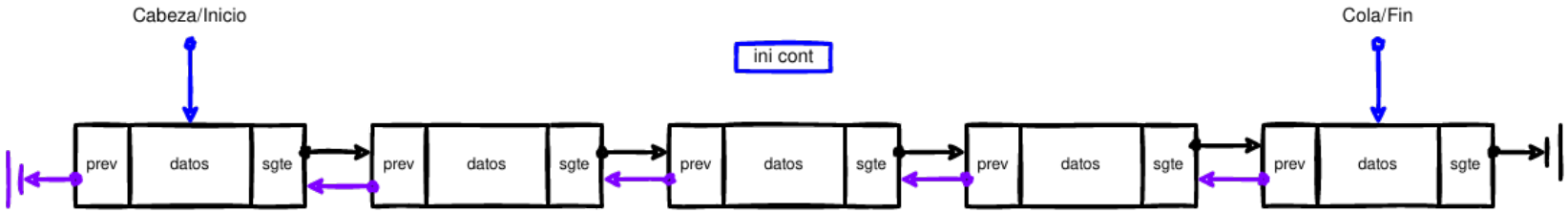
Lista Doblemente Enlazada PRO

Existen otras implementaciones de listas enlazadas simples, donde se incluyen más punteros y variables de control.

C

```c
#include<stdio.h>

typedef struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo enalce.
} nodo_t;

int main(int argc, char **argv){

        nodo_t * inicio = NULL; //pueden usar un struct para estos datos
        nodo_t * final = NULL;
        int largo = 0;
        // Equivalente a lista vacía []
        // ¿Para que sirve esta nueva forma?
        // Implemente las funciones de ésta clase considerando esta forma.

        return 0;
}
```

Lista Doblemente Enlazada PRO



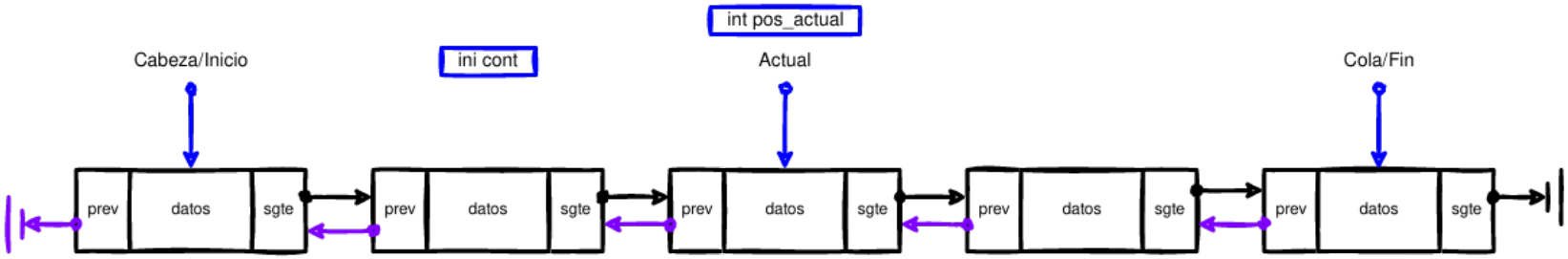
Lista Doblemente Enlazada PRO+

Existen otras implementaciones de listas enlazadas simples, donde se incluyen más punteros y variables de control.

C

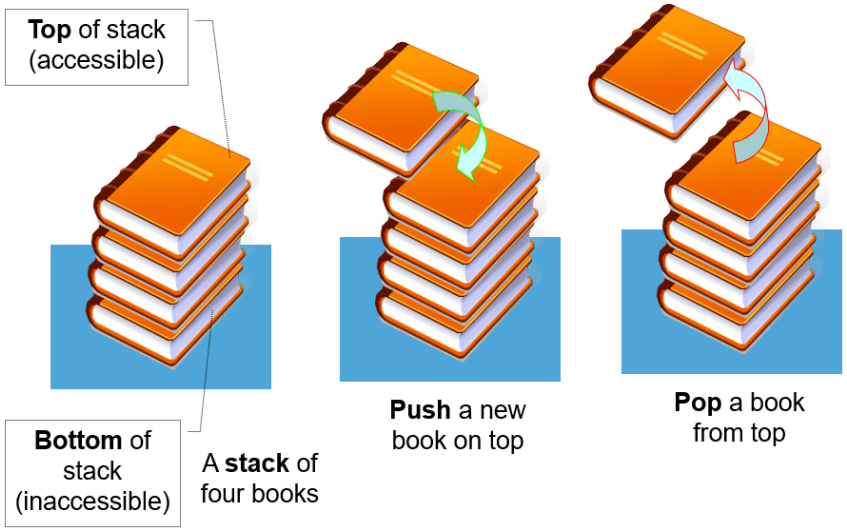
```c
#include<stdio.h>

typedef struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo enalce.
} nodo_t;

int main(int argc, char **argv){

        nodo_t * inicio = NULL; //pueden usar un struct para estos datos
        nodo_t * final = NULL;
        nodo_t * actual = NULL;
        int largo = 0;
        int pos_actual = 0;
        // Equivalente a lista vacía []
        // ¿Para que sirve esta nueva forma?
        // Implemente las funciones de ésta clase considerando esta forma.

        return 0;
}
```

Lista Doblemente Enlazada PRO+



Pilas o Stacks

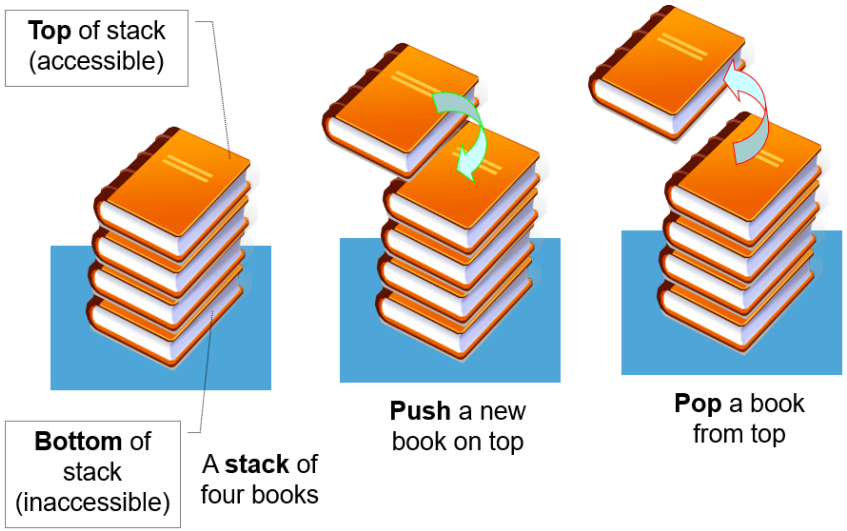
La pila o stack es estructura de datos que permite operar solo en uno de sus extremos. Es de tipo **LIFO: Last In, First Out**. Puede ser implementada con arreglos o listas.



Aplicaciones:

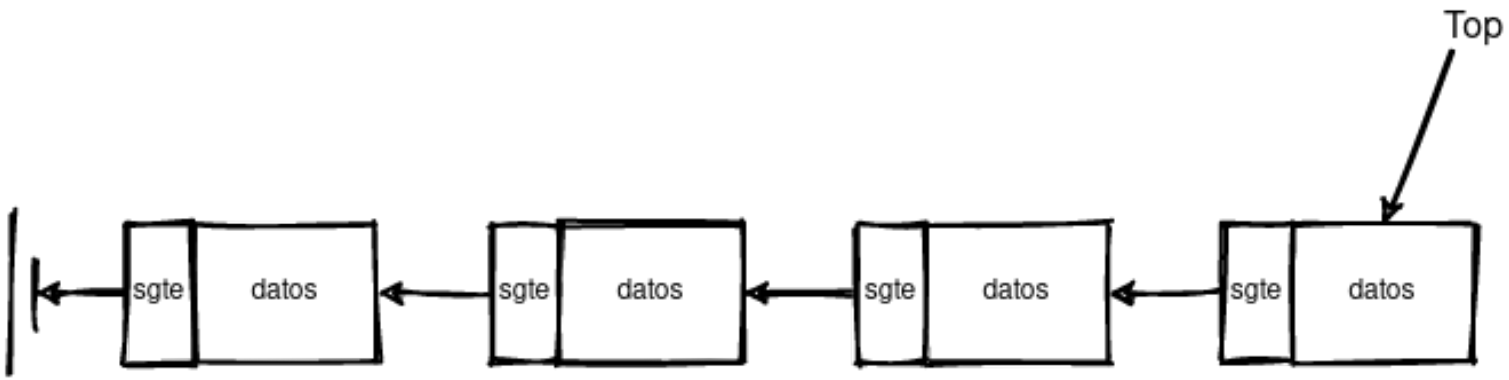
- Control+Z o Control+Y/Control+Mayus+Z: Operaciones de Desahcer o Rehacer (undo/redo).
- Compilación: Formulas matemáticas en notacion de primer orden, i.e. $(x\ 1\ (+\ 2\ 3)) = x\ 1\ +\ 2\ 3$
- Busqueda en árboles: Por ejemplo Búsqueda en Profundidad (Deep-first Search o DFS).

Pilas o Stacks: Interfaz

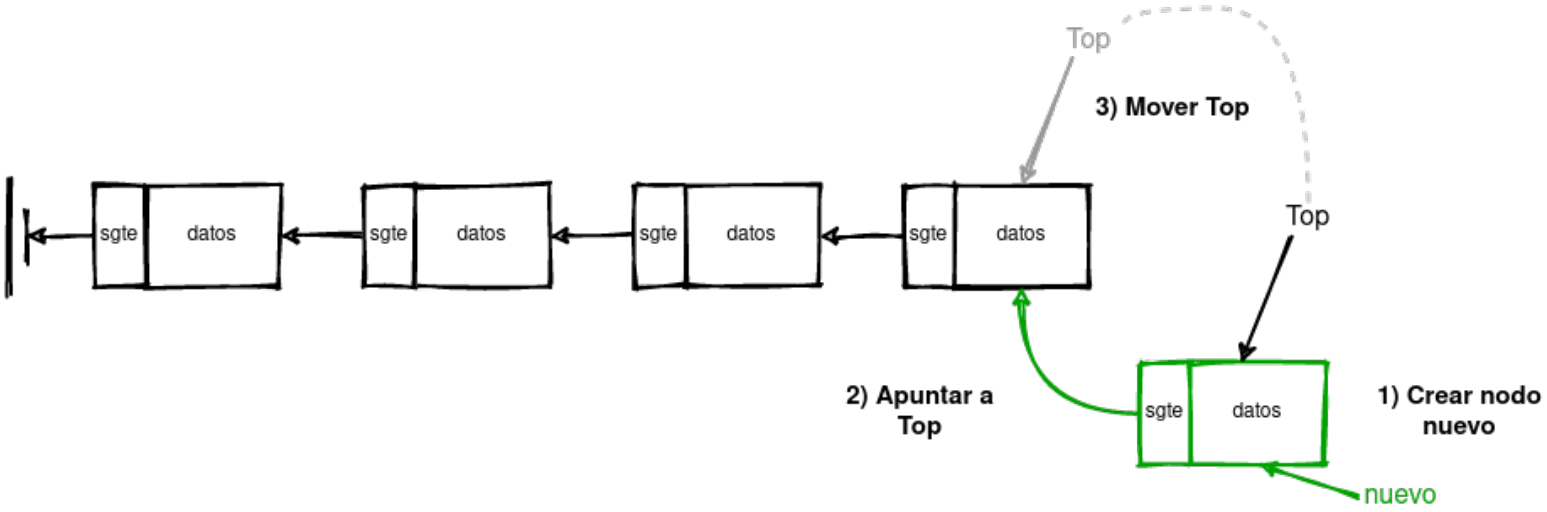


- `push()`: Ingresa un nuevo elemento en la pila.
- `pop()`: Elimina el elemento superior de la pila.
- `peak()`: Devuelve el valor del primer elemento en la pila.
- `is_empty()`: Revisa si la pila está vacía.
- `is_full()`: Revisa si la pila está llena (solo para tamños fijos).
- ¿Cómo se implementan?

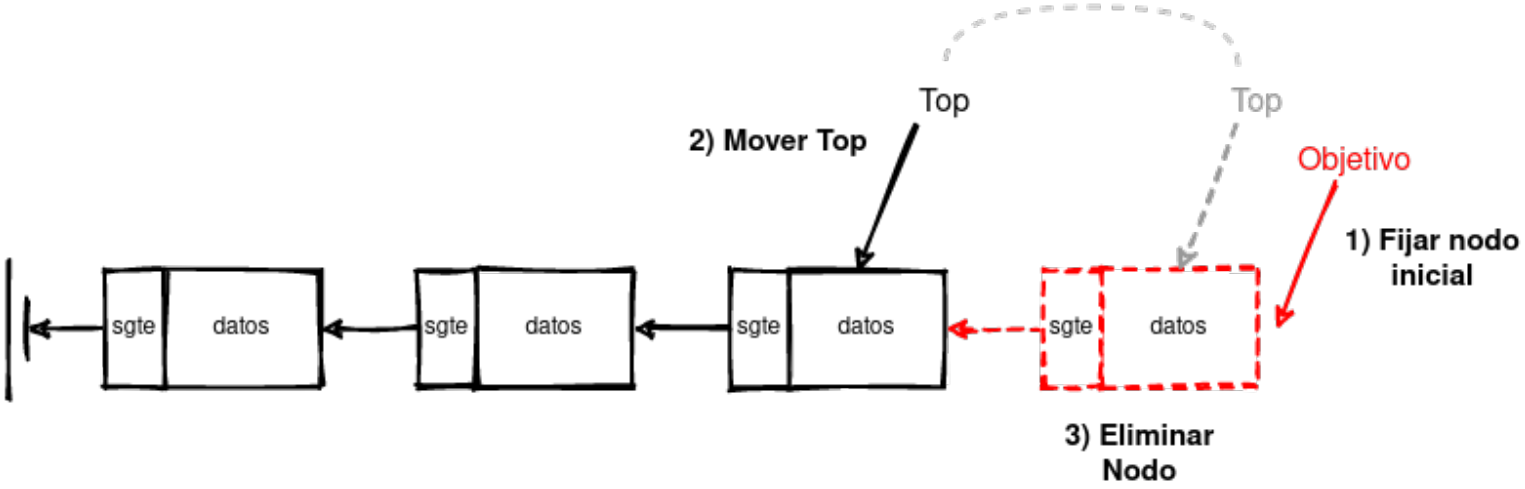
Pilas o Stacks



Pilas o Stacks: push()

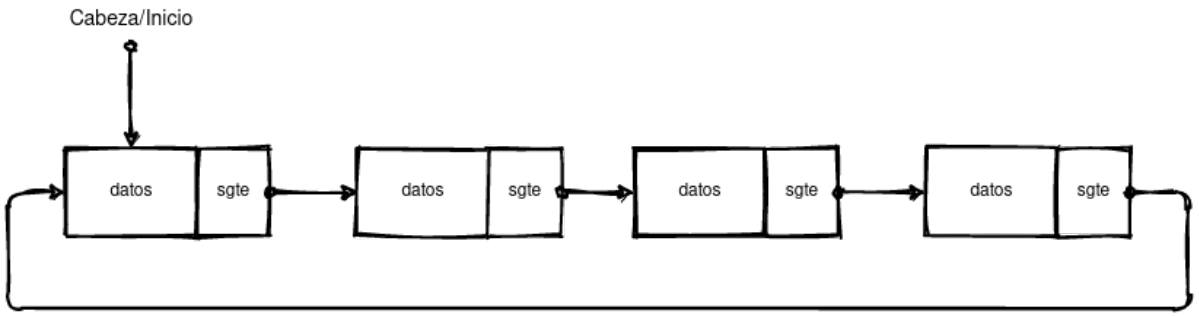


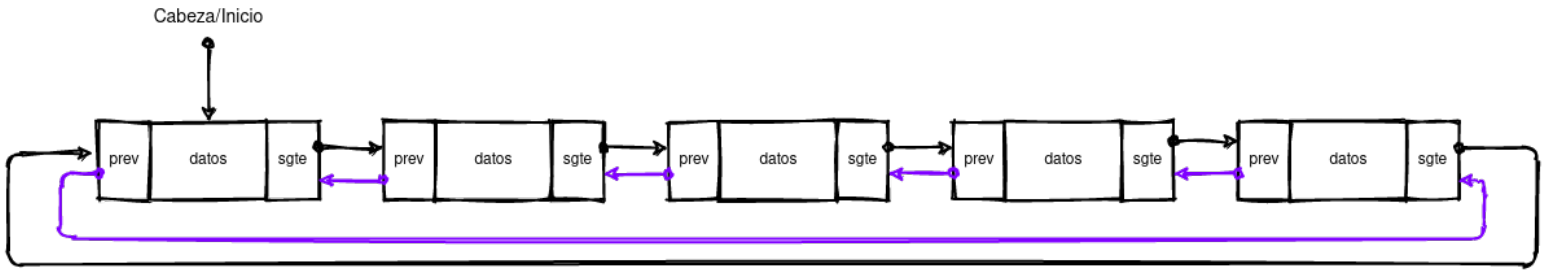
Pilas o Stacks: pop()



Listas Circulares

La lista circular es una variación de la lista simple/doble enlazada en la cual, el primer elemento está conectado con el último elemento, creando así una estructura circular.





Listas Circulares

Ventajas:

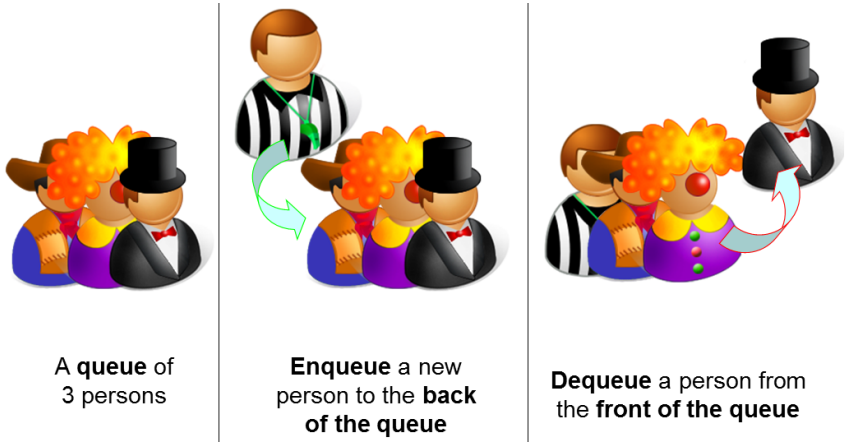
- Cualquier nodo puede ser la cabezera o final.
- Si una estructura tiene dos punteros, ahora solo es necesario uno.
- Útil para lecturas reiterativas de elementos.

Aplicaciones:

- Colas o Queues.
- Buffers circulares dinámicos y estáticos.
- Planificacion de ejecución de componentes de CPU.
- Todo tipo de round robin scheduling.

Colas o Queues

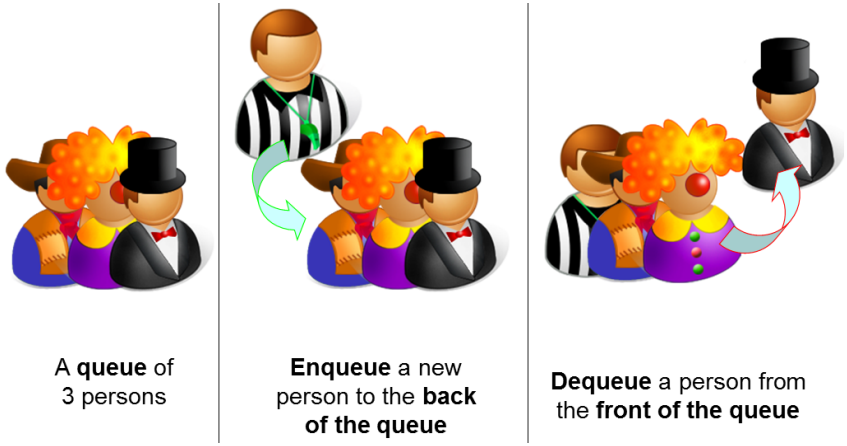
La cola o queue es una estructura de datos que permite operar en un extremo para entrada y en otro extremo para salida. Simula el comportamiento de una fila o cola. Es de tipo **FIFO: First In, First Out**. Puede ser implementada con arreglos o listas.



Aplicaciones:

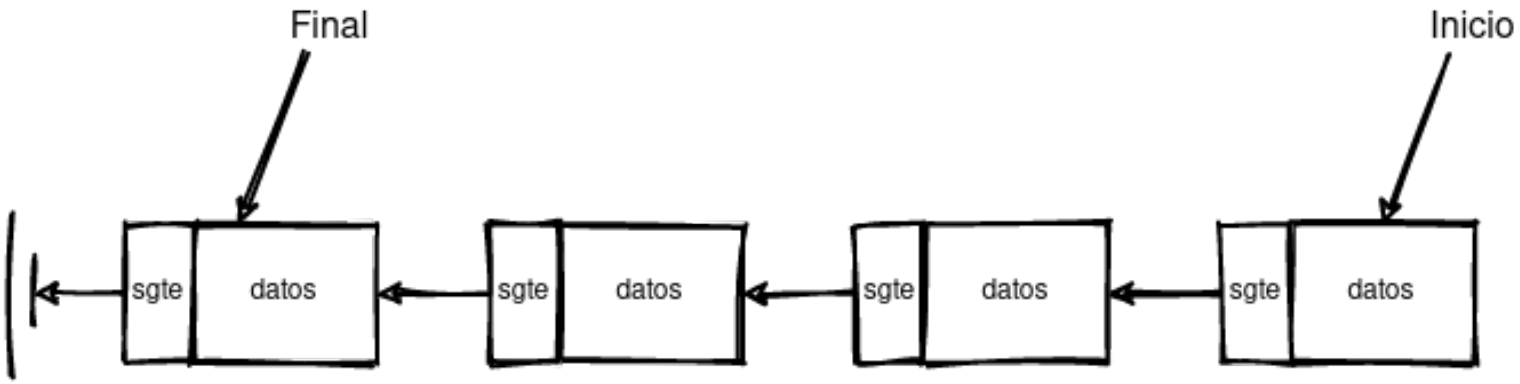
- Planificación de recursos. Priorización.
- Colas de impresión.
- Comunicación asincrónica e interrupciones de sistema.
- Teoría de Colas.

Pilas o Stacks: Interfaz

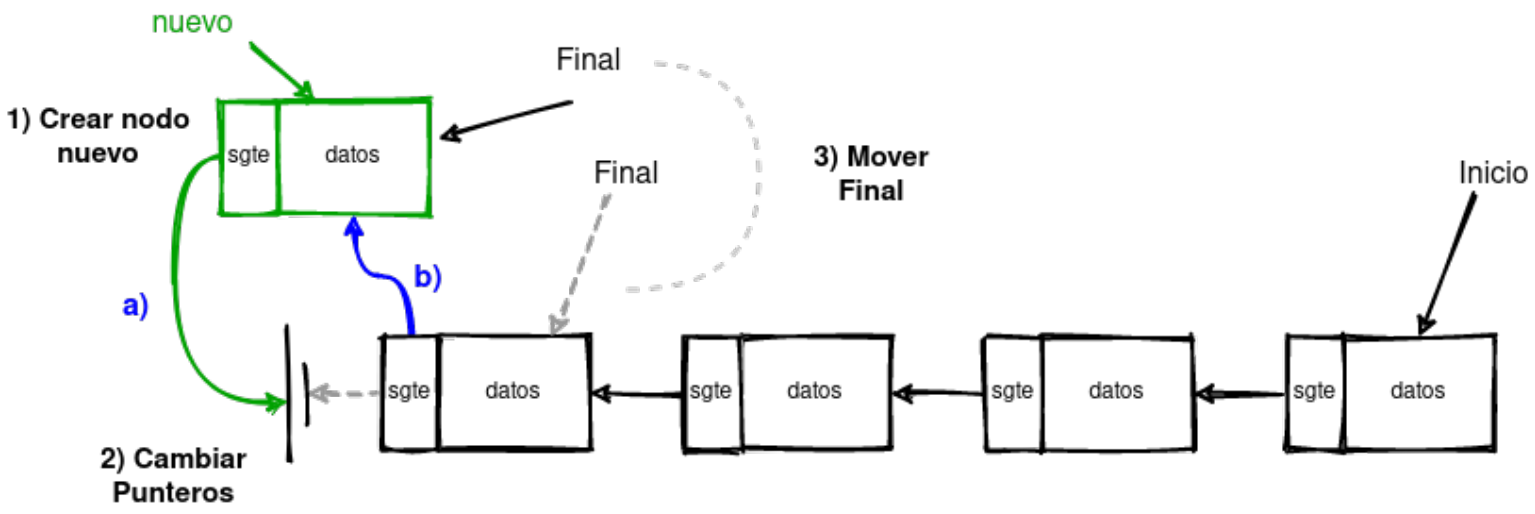


- `enqueue()`: Ingresa un nuevo elemento al final de la cola.
- `dequeue()`: Elimina el primer elemento de la cola.
- `size()`: Devuelve el tamaño de la queue.
- `check()`: Devuelve el valor de un elemento de la cola.
- `is_empty()`: Revisa si la queue está vacía.
- `is_full()`: Revisa si la queue está llena (solo para tamños fijos).
- ¿Cómo se implementan?

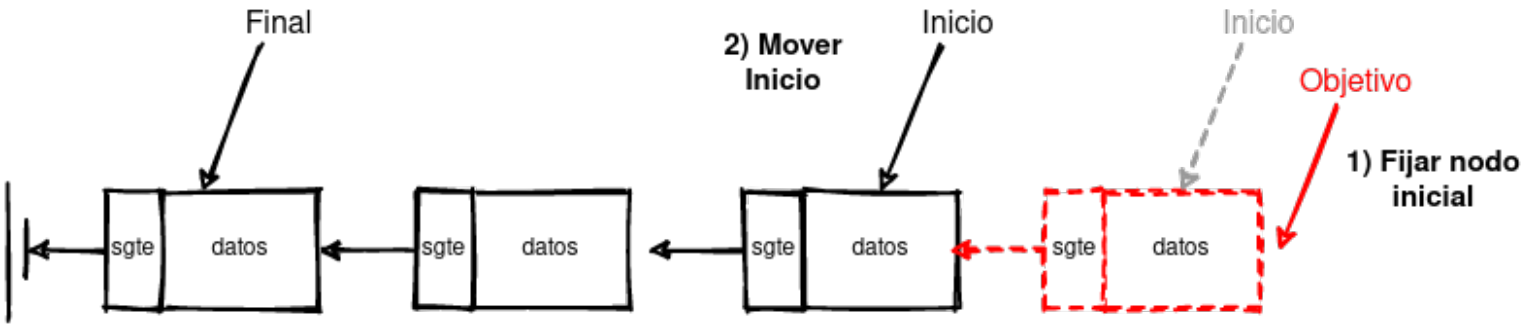
Colas o Queues



Colas o Queues: enqueue()

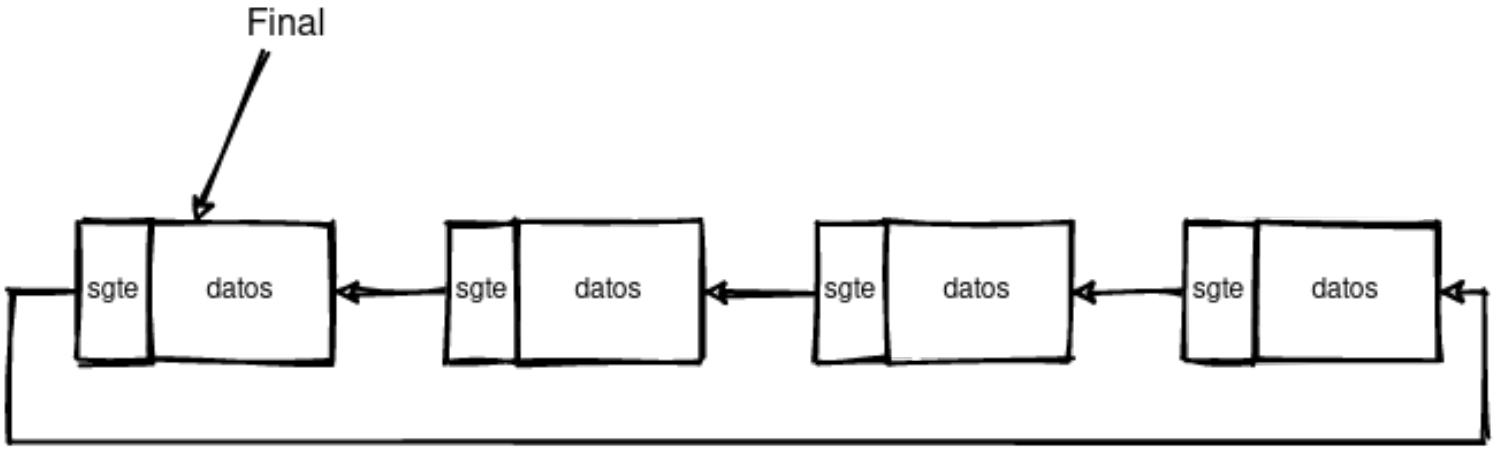


Colas o Queues: dequeue()

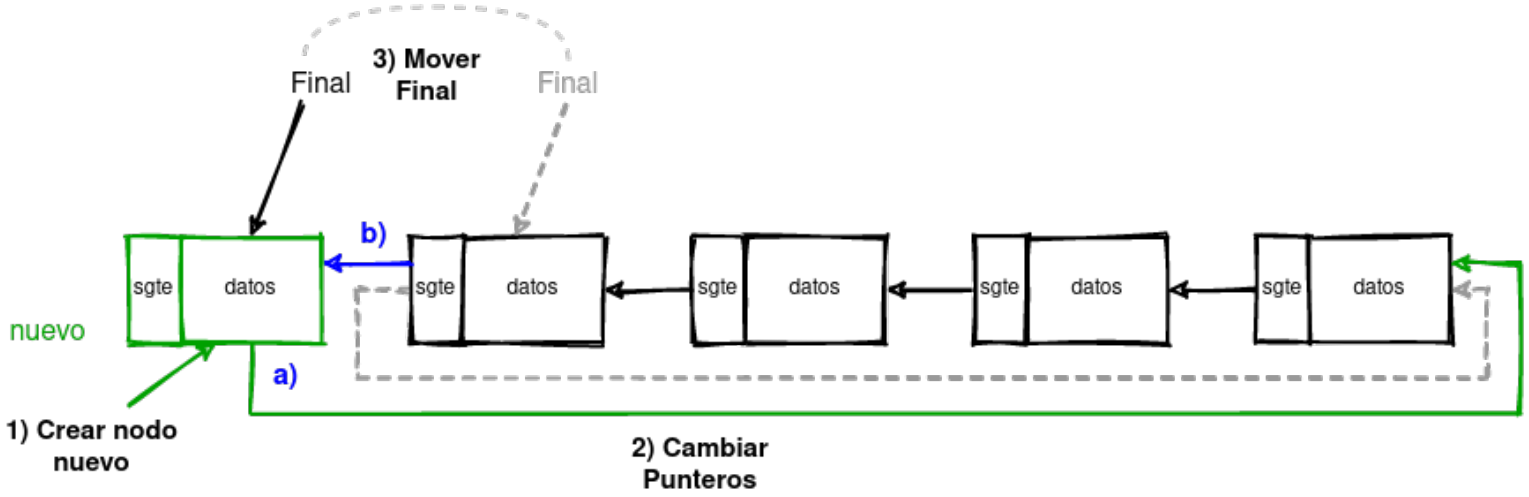


Colas o Queues Circulares

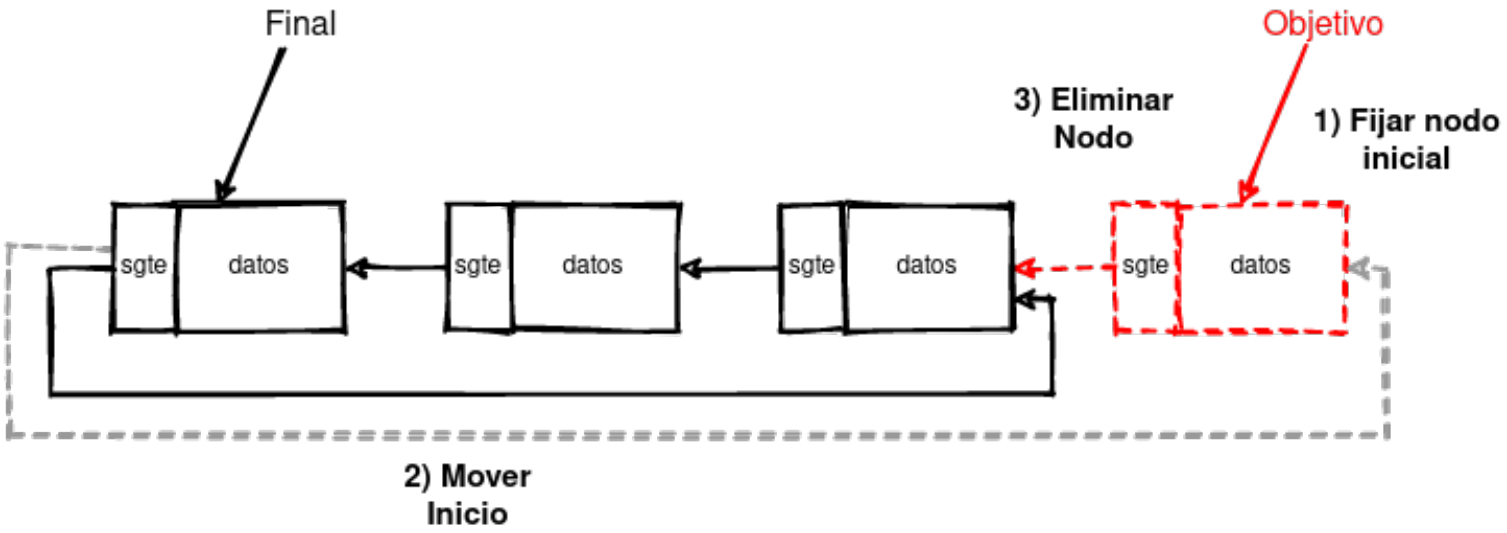
Es una fila o cola que se implementa utilizando estructuras de datos circulares, i.e., listas enlazadas circulares.
Ventaja principal, necesita sólo puntero para ser manejada, en la posición final. No hay necesidad de recorrer toda la fila para poder hacer cambios.



Colas o Queues: enqueue()



Colas o Queues: dequeue()



Desafío

Desarrolle un programa que:

- Invierta una palabra utilizando pilas.
- Inveirta una palabra usando queues.
- Invierta una lista utilizando stacks.
- Invierta una lista utilizando colas.